CogMaPOI: A Cognition-Inspired Multi-Agent Framework for Continual Next POI Recommendation with Episodic Memory and Metacognitive Reflection

A report submitted in partial fulfilment of the requirements

for the award of the degree of

B.Tech in Computer Science and Engineering

by

Suguna Sri, Chilukala Amrutha
(Roll No: 122CS0004, 122CS0065)

Under the Guidance of

Dr. K.E. Srinivasa Desikan

Assistant Professor

Dept. of CSE, IIITDM Kurnool

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

DESIGN AND MANUFACTURING KURNOOL

April 2026

Evaluation Sheet

Title of the Project:

CogMaPOI: A Cognition-Inspired Multi-Agent Framework for Continual Next POI Recommendation with Episodic Memory and Metacognitive Reflection

Name of the Students:

Suguna Sri (122CS0004)

Chilukala Amrutha (122CS0065)

Examiner(s):

_____

Supervisor:

_____

Head of the Department:

_____

Date:

Place:

Declaration

We, **Suguna Sri** and **Chilukala Amrutha**, with Roll No: **122CS0004, 122CS0065** hereby declare that the material presented in the Project Thesis titled **CogMaPOI: A Cognition-Inspired Multi-Agent Framework for Continual Next POI Recommendation with Episodic Memory and Metacognitive Reflection** represents original work carried out by us in the **Department of Computer Science and Engineering** at the **Indian Institute of Information Technology Design and Manufacturing, Kurnool** during the years **2025 − 2026**.

With our signatures, we certify that:

- We have not manipulated any of the data or results.
- We have not committed any plagiarism of intellectual property. We have clearly indicated and referenced the contributions of others.
- We have explicitly acknowledged all collaborative research and discussions.
- We have understood that any false claim will result in severe disciplinary action.
- We have understood that the work may be screened for any form of academic misconduct.

Date: Students' Signatures

In my capacity as supervisor of the above-mentioned work, I certify that the work presented in this Report is carried out under my supervision, and is worthy of consideration for the requirements of B.Tech. Project work.

Advisor's Name: **Dr. K. E. Srinivasa Desikan** Advisor's Signature

Abstract

Next point-of-interest (POI) recommendation has historically been treated as single-pass prediction: each output is produced independently, with no channel for test-time observations to influence what follows. This design misses three faculties that drive human spatial decisions: stable long-term preferences, awareness of recent detours, and reflection on past misjudgements. CogMaPOI closes this gap through a five-agent LLM architecture that operationalises all three as separately maintained, mutually interacting components of a continual inference loop. A fixed identity prior encodes stable mobility preferences; episodic memory accumulates natural-language summaries of test-session behaviour; and a metacognitive reflection record evaluates completed predictions and issues targeted corrections for subsequent ones. Before the loop starts, a Retrieval Agent puts together a candidate set tailored to each user, and at every episode a Reranker brings the whole cognitive state together to produce the final ranked output. The state evolves purely through structured prompt updates, without touching any model parameters. On three Foursquare datasets, CogMaPOI shows relative MRR improvements of up to 89.35% compared to the strongest baseline for each metric. The ablation results also indicate that memory and reflection both play important roles, contributing in complementary and non-redundant ways. Even when using a 3B-parameter backbone, the full system continues to outperform the strongest static baseline across all reported metrics. This suggests that the gains come from the architecture itself, rather than simply from increasing model size. Overall, these results present test-time cognitive adaptation as a practical and parameter-free alternative to fine-tuning for location prediction.

Keywords: Next POI Recommendation, Large Language Models, Multi-Agent Systems, Episodic Memory, Metacognitive Reflection, Continual Inference

Acknowledgements

We would like to thank our guide, **Dr. K. E. Srinivasa Desikan**, Assistant Professor, Department of Computer Science and Engineering, for his continuous support and help throughout the project. His guidance and advice have really helped us analyze and carry out the research work more efficiently.

We also thank **Neeli Venkata Siva**, Ph.D. scholar working under Dr. K. E. Srinivasa Desikan, for all his support, guidance, and suggestions on technical inputs. His timely help and advice proved very useful to improve the quality and direction of this work.

Our sincere thanks to **Dr. Srinivas Naik**, Head of the Department, Department of Computer Science and Engineering for his time and for ensuring support in providing a supportive academic environment which facilitated successful execution of this project.

We owe our seniors, families, and friends, as they continuously motivated us for the development work, supported us, and understood every step we took throughout in order to stay focused on this project.

We truly appreciate the encouragement and help provided by the Department of Computer Science and Engineering at the Indian Institute of Information Technology Design and Manufacturing, Kurnool, for establishing a good academic environment and providing essential resources needed to execute this work effectively.

Contents

Evaluation Sheet i

Declaration ii

Abstract iii

Acknowledgements iv

Contents v

List of Figures viii

List of Tables ix

Abbreviations x

1 Introduction 1
- 1.1 Overview . . . 1
- 1.2 Next-POI Recommendation . . . 2
- 1.3 Large Language Models and Multi-Agent Frameworks . . . 2
- 1.4 Cognition-Inspired Spatial Reasoning . . . 3
- 1.5 Origin of Work . . . 4
- 1.6 Problem Statement . . . 5
- 1.7 Objectives . . . 5
- 1.8 Proposed Approach . . . 6

2 Literature Review 7
- 2.1 Deep Learning for Next-POI Recommendation . . . 7
- 2.2 LLM-Based POI Recommendation . . . 9
- 2.3 Multi-Agent Recommendation Frameworks . . . 10
- 2.4 Cognition-Inspired Continual Learning . . . 11
- 2.5 Research Gaps . . . 12

3 Methodology **13**
3.1 Preliminaries . . . 13
3.1.1 Entities and Notation . . . 13
3.1.2 Episode Structure . . . 14
3.1.3 Agent Prompting Function Notation . . . 15
3.2 Problem Formulation . . . 15
3.2.1 Cognitive State Representation . . . 15
3.2.2 Per-Episode Prediction Task . . . 16
3.2.3 State Transition . . . 17
3.3 Proposed Methodology . . . 17
3.3.1 Profiler Agent: Identity Construction . . . 18
Behavioural summary (b_u). . . . 18
Preferred categories (c_u). . . . 18
Routine patterns (r_u). . . . 19
Exploration style (e_u). . . . 19
Temporal habits (τ_u). . . . 19
Strategy hint (s_u). . . . 19
3.3.2 Retrieval Agent: Candidate Construction . . . 19
3.3.3 Memory Agent: Episodic Consolidation . . . 20
3.3.4 Reflection Agent: Metacognitive Reasoning . . . 21
3.3.5 Reranker Agent: Inference and Integration . . . 22
3.3.6 The Continual Episodic Inference Loop . . . 22
Temporal causality. . . . 22
Progressive enrichment. . . . 22
Parameter-free adaptation. . . . 23
Evaluation fairness. . . . 23

4 Implementation Details **24**
4.1 Hardware and Software . . . 24
4.2 Model Configuration . . . 24
4.3 Datasets . . . 25
4.4 Data Preprocessing and Episode Construction . . . 25
4.5 Evaluation Metrics . . . 25
4.6 Baselines and Comparison Protocol . . . 26

5 Results and Analysis **27**
5.1 Comparative Performance Analysis . . . 27
5.1.1 The MRR-versus-HR Asymmetry . . . 27
5.1.2 Dataset-Level Analysis . . . 28
5.2 Ablation Study . . . 29
5.2.1 Memory as Broad Contextual Enrichment . . . 30
5.2.2 Reflection as a Top-Rank Corrector . . . 31
5.2.3 Synergistic Effect of Memory and Reflection . . . 31
5.3 LLM Backbone Sensitivity . . . 32

5.4 Retrieval Stage Analysis 33
5.4.1 Candidate Set Size 33
5.4.2 Transition Window Size 34
5.5 Discussion and Key Takeaways 35

6 Conclusion and Future Work **37**
6.1 Conclusion 37
6.2 Future Work 38

List of Figures

3.1 CogMaPOI architecture: five-agent pipeline with continual episodic inference and cognitive state updates. 18

5.1 Recall@K and MRR vs. candidate set size K (NYC, L=2). 34
5.2 Recall@K and MRR vs. transition window size L (NYC, K=125). 35

List of Tables

3.1 Agent-level prompting function notation. 15

4.1 Summary of key statistics across the three Foursquare datasets 25

5.1 Performance comparison (%). N@k = NDCG@k. denotes relative improvement over the best baseline. NYC TKY CA Model HR@5 HR@10 28
5.2 Ablation on NYC (%). Relative decrease from CogMaPOI in parentheses. . 30
5.3 Backbone sensitivity on NYC (%). 32
5.4 Effect of candidate set size K on NYC (L=2, %). 34
5.5 Effect of transition window size L on NYC (K=125, %). 35

Abbreviations

POI	**P**oint **o**f **I**nterest
LLM	**L**arge **L**anguage **M**odel
MRR	**M**ean **R**eciprocal **R**ank
HR	**H**it **R**ate
NDCG	**N**ormalised **D**iscounted **C**umulative **G**ain
NYC	**N**ew **Y**ork **C**ity (Foursquare dataset)
TKY	**T**o**k**yo (Foursquare dataset)
CA	**C**alifornia (Foursquare dataset)
NF4	**N**ormal**F**loat **4**-bit quantisation
FP16	**F**loating **P**oint **16**-bit precision

Dedicated to our professors, parents, and friends, whose unwavering support, encouragement, and belief in us have been the foundation of all our achievements...

Chapter 1

Introduction

1.1 Overview

The question of where a person is likely to go next has obvious practical weight. Navigation assistants, transit planning tools, venue discovery platforms and urban mobility analytics all lean, in one way or another, on some model that can take a user's movement history and turn it into a useful guess about the next destination. Phones now record check-ins with a level of detail that would have been hard to imagine a decade ago, and as that data has piled up, getting personalised location prediction right has become one of the more interesting problems sitting between machine learning and spatial computing.

At the centre of this problem is the notion of a point of interest (POI). A POI is any named, visitable place that a user can check into, a coffee shop, a train station, a park, a museum, and so on. Given a sequence of past check-ins, the next-POI recommendation task is to name the single destination the user is most likely to head to next. On paper the problem sounds neat, but in practice it has a lot of moving parts. The next visit depends not only on where the user has already been; it depends on who they are, what hour of the day it is, how the last few hours have actually unfolded, and whether their current pattern is drifting away from their usual routines. Building something that handles all of this at once, and does it without being allowed to retrain at deployment time, is the problem we tackle in this thesis.

1.2 Next-POI Recommendation

In formal terms, next-POI recommendation takes the user's visit history, a sequence of timestamped check-ins at named places, and ranks candidate destinations in decreasing order of how likely the user is to go there next. The usual framing is supervised sequence prediction: we train on historical trajectories and evaluate on held-out test sequences.

The problem has been on the community's radar for more than a decade. Early attempts used recurrent networks to pick up on the temporal ordering in check-in sequences. After that came self-attention architectures borrowed from language modelling, then graph-based encoders that try to express how venues relate to each other in space, and most recently generative models that treat preference drift as something continuous in time. Each family has pushed benchmark accuracy a little further. But one thing has stayed fixed across all of them: once training ends, the parameters are locked. Nothing observed during evaluation can change how the model behaves. If a user's habits drift between training and evaluation, and real users drift all the time, there is simply no mechanism by which the model can pick that up.

1.3 Large Language Models and Multi-Agent Frameworks

The arrival of large language models (LLMs) changed what was possible here. LLMs pre-trained on huge text corpora and then instruction-tuned can read natural-language descriptions of a user's behaviour, reason about temporal and categorical patterns, and produce a ranked output without any task-specific fine-tuning. Applied to next-POI recommendation, the usual trick is to turn a check-in history into a structured prompt and let the model lean on its pre-existing semantic knowledge of venues and places to do the reasoning.

Multi-agent frameworks take this a step further. Instead of asking a single model call to simultaneously pull out user preferences, reason about spatial transitions and emit a ranking, a multi-agent pipeline splits the job: each concern is handed to a dedicated agent with its own focused prompt and its own output format. The end result is a more modular pipeline, one that is easier to interpret, and easier to improve piece by piece — than a monolithic single-call system.

For the next-POI setting, this pattern is a reasonably natural fit. The prediction is compositional: you need to know who the user is, what has been going on recently, what the local spatial situation suggests, and then blend those signals into a single ranking.

Current multi-agent systems do show that splitting these jobs across specialised agents gives better results than a single-agent call. What they do not do is carry anything across predictions. The next episode starts from scratch; nothing from the previous prediction is available, and every ranking is produced as if the system had never seen the user before.

1.4 Cognition-Inspired Spatial Reasoning

Human navigation is not stateless. Spatial decisions pull in a few different kinds of knowledge at once, each of which works on its own timescale and does its own job.

One kind is the long-term picture of personal preferences and habits: which neighbourhoods someone tends to end up in, which kinds of venues they gravitate toward, and roughly how the day is structured from one week to the next. This is the layer that barely shifts across weeks or months.

Another kind is short-term episodic awareness, the ability to fold in what has just happened: where we are at this moment, whether the surroundings are familiar, and whether the last few stops have started to drift away from the usual pattern. This layer is updated more or less continuously, and it matters most when behaviour starts diverging from the established one.

The third one is a little more complex to define since it requires introspection. Metacognition is when you realize that a well-known strategy is not effective and you need to find another route. If we make two mistakes following a specific rule of thumb, chances are that we would try a new method after the third time. Here, the objective is not only to recall past experiences but also to assess whether our thinking process is actually working as intended.

As such, these three dimensions interweave smoothly as we navigate. But no current next point of interest recommendation engine manages to integrate the three simultaneously. Long-term preferences can be incorporated through historical training, but short-term episodic memory and metacognitive adaptation must be inferred from what happens during inference. Since a fixed parameter model cannot adapt to new information, there is a clear gap between human spatial cognition and the capability of current navigation systems.

1.5 Origin of Work

The initial inspiration for this research was derived from an observation about next POI systems. The good-performing next POIs usually have high values of Hit Rate but poor Mean Reciprocal Rank. It means that the required destination is present among top-10 destinations but rarely takes first place. It proves that the problem is not in retrieving data, but in its ranking.

The deeper analysis of the ranking mistakes revealed another factor behind this phenomenon. For the same user, errors were directional rather than random. The system would consistently favour one category of venue over another, or consistently under-weight a type of temporal cue, producing the same kind of mistake repeatedly. This is precisely the kind of systematic error that a person would notice and adjust for after the first or second occurrence. The system had no such capacity; it applied the same learned function to every query without any awareness of whether its recent predictions had been accurate.

This raised a design question: could a recommendation system carry information forward across successive predictions, specifically about what the user has been doing during the current session and whether the system's own recent predictions were correct? The difficulty was that existing approaches to adaptive modelling require gradient updates, which means maintaining a training pipeline at inference time, which is impractical in real-time deployment.

The key insight came from recognising that large language models, when given structured natural-language context, can reason from that context without any parameter changes. A cognitive state maintained as natural language inside a prompt is updated by rewriting the prompt, not by running backpropagation. This made it possible to implement both short-term episodic awareness and metacognitive error correction through structured prompt engineering within a multi-agent loop, entirely at inference time and with no modification to model weights.

CogMaPOI is the realisation of these decisions. The experiments in this thesis test whether this architecture, built on top of a consumer-grade instruction-tuned LLM, can consistently outperform both parametric deep learning baselines and the strongest existing multi-agent systems across diverse benchmark datasets.

1.6 Problem Statement

The standard deployment pipeline for next-POI recommendation has no channel through which anything that happens at test time can influence what the system does next. Whatever the model learned during training is the only thing it will ever apply, no matter how far the user's evaluation-time behaviour drifts from what it saw in the training trajectory. A more sensible formulation would treat the test session itself as evidence, build up that evidence step by step, and let the system's own past mistakes inform future decisions.

The problem addressed in this thesis is stated as follows:

> *To develop a cognition-inspired multi-agent framework that reformulates next-POI recommendation as a continual episodic inference process, enabling parameter-free test-time adaptation through identity priors, episodic memory consolidation, and metacognitive reflection.*

The cognitive state has to carry three qualitatively different kinds of information at the same time. The first captures stable long-term user preferences learned from past data. The second reflects short-term episodic evidence of what the user has done in the current session. The third keeps track of how well the system's earlier predictions performed, and uses that information to guide the episodes that follow.

1.7 Objectives

The specific objectives of this thesis are:

1. To formalise next-POI recommendation as continual episodic inference, where a structured cognitive state evolves across prediction episodes without any parameter updates to the underlying model.
2. To design a five-agent LLM architecture that operationalises long-term preference modelling, short-term episodic adaptation, and metacognitive error correction as distinct, interacting state components in a strictly causal inference loop.

3. To rigorously evaluate the framework against thirteen baselines spanning transformer-based, graph-based, generative, LLM-based, and multi-agent methods across three Foursquare benchmark datasets, measuring performance on both coverage metrics and the ranking-sensitive metric MRR.
4. To isolate the individual contributions of episodic memory and metacognitive reflection to prediction quality through controlled ablation, and to determine whether their joint effect exhibits interaction effects that neither component produces independently.
5. To assess the robustness of the framework across LLM backbones ranging from 3B to 8B parameters, and to determine whether the performance advantage is structural in origin rather than contingent on model scale.

1.8 Proposed Approach

CogMaPOI tackles the problem with a five-agent pipeline. Across a three-episode test window, it keeps three state components running in parallel, each managed on its own.

Before inference starts, a Profiler Agent builds a structured identity prior out of the user's historical trajectory; this prior captures their stable preferences, routine patterns and a synthesised reasoning hint. At roughly the same time, a Retrieval Agent assembles a user-specific candidate set by combining population-level transition statistics with the user's own visit frequency, weighted adaptively based on how much they tend to explore.

At inference time, the test trajectory is chewed through in three episodes of ten check-ins each. After every episode, a Memory Agent updates a running natural-language summary of the user's test-session behaviour, noting category shifts, new spatial areas, and anything that looks like a departure from the routines encoded in the prior. A Reflection Agent then grades the system's prediction for that episode: when the prediction was right, the reasoning strategy that produced it is reinforced; when it was wrong, the agent diagnoses what went wrong and produces a corrective directive for the next episode. On the following step, a Reranker Agent conditions on the current episode, the identity prior, the episodic memory and the accumulated reflection record, and emits the next prediction.

At no point does any agent fine-tune a model. Every piece of adaptation is carried through natural-language state updates inside structured prompts. By the time the system reaches Episode 3, which is the only episode we score, it has already gone through two full rounds of episodic consolidation and metacognitive reflection.

Chapter 2

Literature Review

This chapter puts CogMaPOI in context by walking through the work that motivates it. The review is laid out in four parts: deep learning methods for sequential next-POI recommendation, LLM-based approaches, multi-agent recommendation frameworks, and cognition-inspired continual learning. In each part we look at what the paradigm contributed and, more importantly, at the specific limitation that kept pushing the field toward whatever came next. A closing section then distils the research gaps this work sets out to address.

2.1 Deep Learning for Next-POI Recommendation

The move from collaborative filtering to deep sequential models marked a meaningful shift in how user mobility came to be modelled. From early recurrent models such as LSTM trajectory encoders, it was clear that there is more information encoded in the temporal order of check-ins than what simple co-occurrences would convey. After demonstrating their effectiveness on sequences, self-attention models fairly soon replaced these recurrent models.

Transformers for recommendation. SASRec [1] propose, where self-attention is applied in a unidirectional way on the sequence of items, thus enabling the model to rely on the most contextually relevant check-ins to predict the next one. BERT4Rec [2] go further, by using bidirectional attention and defining the problem as masked item recovery. TiCoSeRec [3] then plugs a gap these two leave open: it adds time-interval-aware augmentation to the transformer backbone, so the gaps between

consecutive check-ins end up as features rather than something the model silently assumes to be uniform. DuoRec [4] goes in another direction and attacks representation degeneration through contrastive learning, using augmented views of the same trajectory to keep the embedding space from collapsing on itself.

Together, these works establish that sequential modelling over check-in histories is a perfectly workable base for next-POI prediction. They also share a limitation, though. Preferences are frozen at training time: once a user's historical trajectory has been digested into the weights, any drift that happens later during the evaluation window has nowhere to go.

Graph-based methods. GETNext [5] wraps transformer-based trajectory encoding around a global trajectory flow map, which brings population-level mobility patterns into individual predictions in the form of a graph. CrossDR [6] points out that the spatial and temporal sides of mobility follow different statistical regularities, and disentangles them into separate representation spaces to help cross-context generalisation. MAERec [7] takes a self-supervised angle, using a graph masked autoencoding objective over trajectory data to recover high-order dependencies between POIs. These approaches introduce richer inductive biases than pure sequence models but inherit the same static-deployment issue.

Generative and continuous-time methods. MTNet [8] models user preferences through a mobility tree that organises visited venues by time slot, which makes fine-grained time-of-day reasoning more natural. DiffuRec [9] comes at the problem probabilistically, treating the next destination as a distribution over the candidate space and using a diffusion process to model that distribution. POIGDE [10] takes continuity the furthest, using graph differential equations to track preference dynamics at whatever temporal resolution one likes. These methods make the modelling toolbox more expressive, but none of them breaks the single-pass habit, the prediction is produced once, from the history that happens to be available, and cannot be revised in light of anything that the system sees during evaluation.

A property common to all three sub-families is that venues end up as anonymous numeric identifiers. No model in this paradigm can really reason about what a venue is, how it relates to other venue types semantically, or what role it plays in everyday routines. That representational ceiling, more than any shortage of architectural cleverness, is what eventually held deep sequential models back. Moving past it required a foundation in which broad semantic knowledge about places is already there at inference time, not something

that has to be reconstructed from check-in statistics alone. That is exactly what large language models brought to the table.

2.2 LLM-Based POI Recommendation

The arrival of large language models (LLMs) opened up a new route for next-POI recommendation, one that works by reasoning over mobility data in natural language. Rather than pushing trajectories through as sequences of item IDs, LLM-based methods reformat check-in histories as descriptive prompts, so the semantic knowledge already baked into the pre-trained model can be brought to bear on the task.

LLM4POI [11] was among the first papers to demonstrate this at scale. It takes user trajectories, turns them into structured natural-language prompts, and runs zero-shot LLM inference on the ranking task with no task-specific fine-tuning. The headline finding was that LLMs do recover meaningful predictive signal from these textual descriptions, especially for the kinds of categorical and temporal preferences that are genuinely awkward to express as numeric features. ZeroPOIRec [12] extends this line of work by relying on pre-trained LLMs to make sense of human behaviour from individual visit histories alone; the framework uses specialised profiler and recommender modules to tighten up candidate selection in a zero-shot setting.

Subsequent research efforts have been predominantly dedicated to improving the prompt aspect of the narrative. GNPR-SID [13] proposes to construct semantic location identifiers based on the distillation of venue attributes into concise descriptors that conform well to the structure of the LLM prompt. Similarly, QA-POI [14] presents an approach to solving the problem of spatial recommendations by treating it as a task of question answering that exploits the dialogue capabilities of instruction-tuned LLMs. Some survey papers [15, 16] have provided a broader analysis of the compatibility of LLMs with location-based services; both agree that the most distinctive competitive edge of LLMs over numeric models is their capability of conveying preference information qualitatively.

What all of these approaches fail to account for is the idea of state. The prompts for each prediction attempt are unique; there is no way of drawing on experiences from previous attempts. There is also no means by which the system can look back on past failures. If the system makes an incorrect recommendation about venues at some point along the test's timeline, that failure has no route to affect future attempts.

In addition to the stateless property, there is another drawback associated with the single-model framework. Generating a trustworthy suggestion requires more than one stage of reasoning; it is a combination of qualitatively distinct stages, including generating an accurate representation based on past experience, understanding the current spatial and temporal context, and then making distinctions between viable options. The task of assigning such a multi-faceted job to a single model becomes problematic because the issues involved will be fighting for attention and, as a result, will not receive the careful consideration they deserve. Multi-agent systems were developed precisely for this purpose; give each sub-task to a specialist agent and let it reason about that dimension independently.

2.3 Multi-Agent Recommendation Frameworks

Multi-agent frameworks split the recommendation task into specialised components, allowing different parts of the problem to be handled by dedicated agents. For tasks that involve multiple interacting sub-problems, this kind of setup has consistently performed better than monolithic single-model approaches [17, 18].

MACRec [19] is one of the earlier examples in the general recommendation setting. It organises agents into a role-based hierarchy, coordinators and executors that talk to each other through structured message passing, and showed that splitting the pipeline into distinct reasoning steps, each handled by a specialised agent, produces outputs that are more consistent and easier to interpret than end-to-end approaches on the same model.

CoMaPOI [20] took this idea and applied it directly to next-POI prediction. It bridges structured trajectory data and natural-language reasoning by handing out different pieces of the task, trajectory encoding, contextual analysis, and final ranking, to different agents. The framework gets strong numbers on the Foursquare benchmarks and, at the time of its release, was the result that established multi-agent decomposition as the leading paradigm for this problem.

LMAPOI [21] builds on CoMaPOI's decomposition with a larger agent hierarchy that adds further specialisation around temporal and spatial reasoning. Both frameworks are, today, the strongest multi-agent baselines we have on next-POI recommendation.

That said, CoMaPOI and LMAPOI share the same structural weakness as everything that came before them: there is no unified cognitive state that carries across predictions. Each prediction is made in isolation, with no running record of what the user has been

doing during the current session and no memory of whether the previous prediction was right. The multi-agent architecture resolves the problem of decomposing reasoning *within* a prediction; it says nothing about how information should travel across a sequence of predictions.

That last gap points toward a different corner of the literature. How a system should carry knowledge forward across a sequence of interactions, updating what it knows about the user, and correcting what it knows about its own reasoning, is the kind of question continual learning has been working on for some time. The mechanisms developed in that body of work, and how far they can be pushed without the gradient updates they traditionally rely on, set up the final section of this review.

2.4 Cognition-Inspired Continual Learning

Continual learning looks at the problem of picking up new knowledge without catastrophically forgetting what has already been learned. The link to cognitive science is not accidental, human memory systems are constantly consolidating new experiences on top of stable long-term representations, and continual learning methods are, in large part, attempts to approximate that selective updating computationally.

Gradient Episodic Memory (GEM) [22] frames the problem as a constrained optimisation task. It uses stored examples from earlier tasks as inequality constraints on the gradient updates for new tasks. This provides a guarantee that learning new information will not increase the loss on previously stored knowledge, something standard fine-tuning does not ensure. Error Sensitivity Modulation (ESMER) [23] adjusts the learning signal in a way inspired by neuroscience. The idea is that sensitivity to errors decreases as the error becomes larger. Based on this, ESMER gives more importance to small and consistent errors rather than occasional large ones. As a result, the learned representations change more gradually when the model moves across tasks.

There is also a cognitive science perspective behind these ideas. Pattern separation in the hippocampus leads to stronger encoding when prediction errors are larger [24]. In other words, unexpected outcomes tend to leave stronger and longer-lasting memory traces than expected ones [25]. Schillaci et al. [26] extend this idea to continual learning, suggesting that the size of the prediction error influences which experiences are retained and which internal representations need to be updated to support future learning.

Mapped onto the recommendation problem, these ideas suggest a fairly clean way of doing test-time adaptation. A system that keeps track of its own prediction errors as it goes and uses them to revise its reasoning strategy is, more or less, implementing the same kind of metacognitive error monitoring that cognitive science describes. As far as we know, no prior next-POI recommendation system has actually done this. CogMaPOI does, through the Reflection Agent and the reflection state component associated with it.

2.5 Research Gaps

From the literature we have walked through, four distinct gaps emerge.

Static inference paradigm. All of the reviewed methods produce their predictions from representations that were locked in before evaluation even started. The same learned function gets applied to every query, regardless of what the user actually does once evaluation is underway. Nothing new that happens during the test session can influence any of the predictions that come next.

Lack of explicit episodic memory. None of the existing approaches track what a given user has actually done since evaluation began. Every prediction is drawn from the same historical snapshot, so anything short-term, a change in the kind of venues being visited, a detour into an unfamiliar area, or a departure from an established routine, stays invisible to the system for the entire session.

Absence of prediction error correction. No previous system looks at its own prediction outcomes or adjusts its reasoning on the basis of them. When the same ranking error keeps showing up across several predictions for the same underlying reason, nothing in the system notices the pattern or does anything about it.

No parameter-free test-time adaptation. Continual learning does address session-level adaptation and error-driven correction, but both of its mechanisms require gradient updates during inference, which is simply not practical in a real-time deployment. No prior work has achieved either capability purely through natural-language state updates, without touching the model's parameters after training is over.

CogMaPOI is built to close all four of these gaps at once, through a single five-agent architecture that carries a structured cognitive state across predictions.

Chapter 3

Methodology

This chapter lays out the technical foundation of CogMaPOI in three parts. The first part pins down the notation used in the rest of the thesis. The second part formalises the problem as continual episodic inference and spells out the cognitive state that the system carries across prediction steps. The third part walks through the five agents that actually realise that state, and the causal loop that governs how they talk to each other.

3.1 Preliminaries

3.1.1 Entities and Notation

Let $\mathcal{U} = \{u_1, \ldots, u_N\}$ be a set of N users and $\mathcal{P} = \{p_1, \ldots, p_M\}$ a set of M points of interest. Each POI $p \in \mathcal{P}$ is described by a four-tuple:

$$p = \langle \mathrm{id}_p,\ \mathrm{lat}_p,\ \mathrm{lon}_p,\ \mathrm{cat}_p \rangle, \tag{3.1}$$

where id_p is a unique identifier, $(\mathrm{lat}_p, \mathrm{lon}_p)$ are GPS coordinates in decimal degrees, and cat_p is a semantic category from a fixed taxonomy (e.g., *Food*, *Travel & Transport*, *Nightlife Spot*). A *check-in* records a single visit:

$$x = \langle u,\ p,\ t \rangle, \tag{3.2}$$

where $u \in \mathcal{U}$, $p \in \mathcal{P}$, and t is a Unix timestamp.

For user u, all check-ins sorted chronologically before the test period form the *historical trajectory*:

$$\mathcal{T}_u^H = \{(p_{u,1}, t_{u,1}),\ \ldots,\ (p_{u,n}, t_{u,n})\}, \quad t_{u,i} < t_{u,i+1}. \tag{3.3}$$

The *test trajectory* consists of the final 30 check-ins, following the evaluation protocol of CoMaPOI [20]:

$$\mathcal{T}_u^{test} = \{(p_{u,n+1}, t_{u,n+1}),\ \ldots,\ (p_{u,n+30}, t_{u,n+30})\}. \tag{3.4}$$

The temporal boundary between $\mathcal{T}_u^H$ and $\mathcal{T}_u^{test}$ is strict; no test-period information enters the historical context.

3.1.2 Episode Structure

The test trajectory is divided into three non-overlapping, chronologically ordered episodes of ten check-ins each:

$$\mathcal{T}_u^{test} = \bigcup_{e=1}^{3} \mathcal{E}_u^{(e)}, \qquad |\mathcal{E}_u^{(e)}| = 10. \tag{3.5}$$

Here $\mathcal{E}_u^{(1)}$ holds the earliest ten check-ins, $\mathcal{E}_u^{(2)}$ the middle ten, and $\mathcal{E}_u^{(3)}$ the final ten. At each episode e the system emits a prediction, then updates its internal state before moving on to episode $e{+}1$. The predicted POI for is written $\hat{p}_u^{(e)}$.

Each episode $\mathcal{E}_u^{(e)}$ contains ten check-ins, and we split it into two parts for inference. The first nine, which we write $\mathcal{E}_u^{(e)-}$, are the observable context that the Reranker is actually allowed to condition on at prediction time:

$$\mathcal{E}_u^{(e)-} = \mathcal{E}_u^{(e)} \setminus \{y_u^{(e)}\}, \qquad |\mathcal{E}_u^{(e)-}| = 9. \tag{3.6}$$

The tenth and final check-in is withheld as the ground truth $y_u^{(e)}$, and the Reranker never sees it while making its prediction. After the prediction is emitted, $y_u^{(e)}$ is revealed to the system and folded into the Memory and Reflection updates, so both of those agents operate on the full episode $\mathcal{E}_u^{(e)} = \mathcal{E}_u^{(e)-} \cup \{y_u^{(e)}\}$, which is realistic: once the user has actually been somewhere, the system gets to know where that was.

3.1.3 Agent Prompting Function Notation

Every one of the five agents in CogMaPOI generates its output the same way: by putting together a structured natural-language prompt and sending it to the shared large language model. We write such a prompting call generically as $\mathcal{F}(\cdot)$. Agent-specific instances are distinguished by descriptive subscripts, which are summarised in Table 3.1.

TABLE 3.1: Agent-level prompting function notation.

Symbol	Agent	Role
$\mathcal{F}_{\text{profile}}$	Profiler	Builds the identity prior from $\mathcal{T}_u^H$
$\mathcal{F}_{\text{wt}}$	Retrieval	Derives adaptive fusion weights from $\mathbf{P}_u$
$\mathcal{F}_{\text{mem}}$	Memory	Updates episodic memory after each episode
$\mathcal{F}_{\text{ref}}$	Reflection	Evaluates predictions and produces corrective signals
$\mathcal{F}_{\text{rank}}$	Reranker	Produces the final POI prediction per episode

All five functions run on top of the same underlying model. Whatever distinguishes one agent from another is purely in the prompt structure and the output schema, there are no separate weights anywhere, and no agent-specific fine-tuning.

3.2 Problem Formulation

The main departure from earlier work is that next-POI prediction is reframed here. Instead of a stateless, single-pass inference problem, we treat it as a *continual episodic inference* problem, sitting on top of three cognitive abstractions: identity (the stable, long-term preferences of the user), episodic memory (the behavioural patterns that have just shown up in the current session), and metacognitive reflection (an evaluation of past reasoning, where strategies that worked are reinforced and the ones that failed are revised).

3.2.1 Cognitive State Representation

At every episode $e \in \{1, 2, 3\}$, the system carries a structured cognitive state for user u:

$$\mathcal{S}_u^{(e)} = \left(\mathbf{P}_u,\ \mathcal{M}_u^{(e)},\ \mathcal{R}_u^{(e)}\right). \tag{3.7}$$

The three components hold qualitatively different kinds of knowledge, and each one updates on its own timescale.

$\mathbf{P}_u$ is the *identity prior*, a structured representation of the user's stable, long-term mobility preferences. It is built once from $\mathcal{T}_u^H$ and is then held fixed for the rest of inference. Its role is to anchor the predictions to established habits, so that the system does not slowly drift toward generic, population-level behaviour as the session goes on.

$\mathcal{M}_u^{(e)}$ is the *episodic memory*. It is a running natural-language summary of what the user has been doing during the current test session, initialised to $\mathcal{M}_u^{(0)} = \varnothing$. This is the component that picks up short-term behavioural shifts that may not line up with the baseline encoded in $\mathbf{P}_u$. Updates to the memory are unconditional on prediction correctness, because the record of where the user actually went is useful regardless of whether the previous prediction happened to be right.

$\mathcal{R}_u^{(e)}$ is the *metacognitive reflection record*, the system's own reading of how well it has been predicting, across the episodes it has already completed. It too starts out empty: $\mathcal{R}_u^{(0)} = \varnothing$. The episodic memory tracks what the user did; the reflection record tracks how the system reasoned. A correct prediction reinforces whatever strategy produced it; an incorrect prediction gets a targeted diagnosis and a corrective directive for the next episode. This component has no counterpart in any earlier next-POI recommendation system.

3.2.2 Per-Episode Prediction Task

Given a candidate set $\mathcal{C}_u \subseteq \mathcal{P}$ assembled before inference begins, the prediction at episode e is:

$$\hat{p}_u^{(e)} = \mathcal{F}_{\text{rank}}\Big(\mathcal{E}_u^{(e)-},\ \mathcal{S}_u^{(e-1)},\ \mathcal{C}_u\Big)\,. \tag{3.8}$$

The Reranker conditions on $\mathcal{E}_u^{(e)-}$, i.e. only the nine observable check-ins of the episode; the held-out ground truth $y_u^{(e)}$ is never seen at prediction time. Conditioning on $\mathcal{S}_u^{(e-1)}$ keeps the temporal causality strict: anything observed during episode e cannot influence the episode-e prediction itself, and is only folded in through the state update that follows. The candidate set narrows the full search space down to something tractable; its construction is described in Section 3.3.2.

3.2.3 State Transition

Following each prediction, the cognitive state transitions:

$$\mathcal{S}_u^{(e-1)} \longrightarrow \mathcal{S}_u^{(e)}. \tag{3.9}$$

The identity prior stays put throughout, which protects the long-term preference information from getting washed out by short-term test-session noise. The episodic memory and the reflection record are updated by their respective agents, using the newly observed check-ins together with the latest prediction outcome. All of the state evolution happens through natural-language transformations inside LLM prompts; at no point is any gradient computed or any parameter changed.

3.3 Proposed Methodology

CogMaPOI is made up of five LLM-based agents arranged in a strictly causal loop. The Profiler and Retrieval agents run once per user before inference begins; the Memory, Reflection and Reranker agents run once per episode. Figure 3.1 shows the full pipeline.

The design is driven by a specific observation about human spatial cognition. People get around by using stable long-term habits, an awareness of where they have been recently, and the ability to recognise and correct their own errors. CogMaPOI gives one dedicated agent to each of these three faculties and keeps each faculty's state in its own component, so that updating one does not interfere with the logic of another.

All of the agents share a single instruction-tuned LLM, with no agent-specific fine-tuning anywhere. Any performance difference we can attribute to the framework (see Chapter 5) therefore has to come from the architectural design and the cognitive state structure, not from some parametric advantage picked up during task-specific training. Using a single shared model also keeps the whole system deployable on consumer-grade hardware, which the experimental setup in Chapter 4 makes explicit.

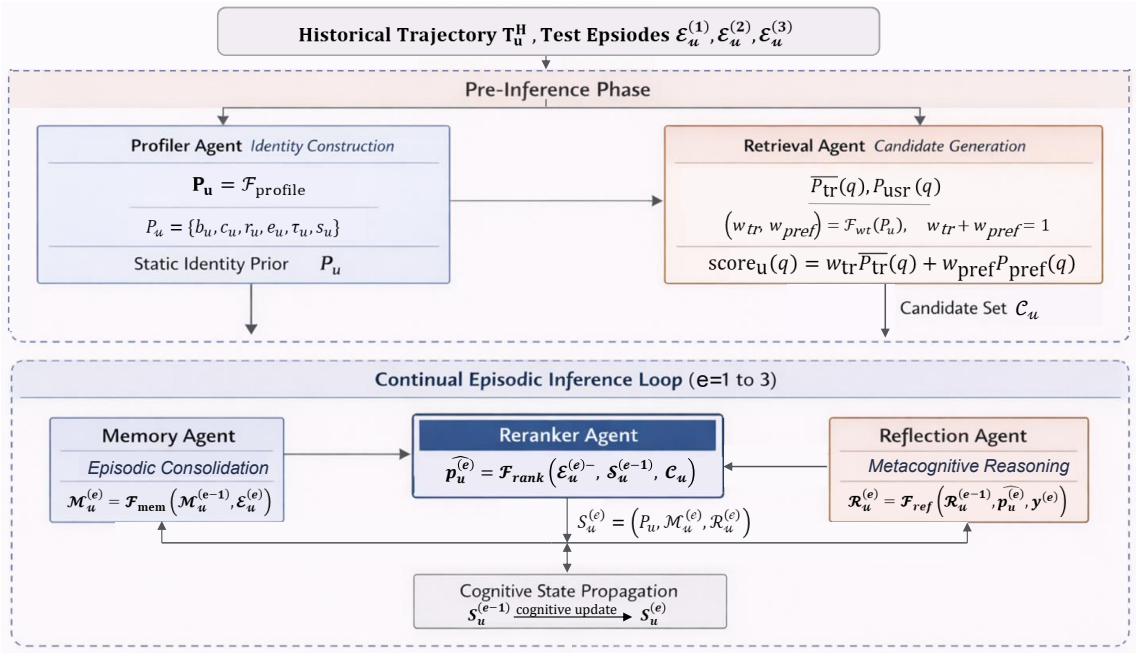

FIGURE 3.1: CogMaPOI architecture: five-agent pipeline with continual episodic inference and cognitive state updates.

3.3.1 Profiler Agent: Identity Construction

The Profiler runs once per user before any inference begins. It takes the full historical trajectory $\mathcal{T}_u^H$ as input and produces the identity prior $\mathbf{P}_u$ as a structured JSON object:

$$\mathbf{P}_u = \mathcal{F}_{\text{profile}}(\mathcal{T}_u^H) = \{b_u,\ c_u,\ r_u,\ e_u,\ \tau_u,\ s_u\}. \tag{3.10}$$

The six fields collectively characterise the user at a level of abstraction that is both human-readable and directly usable by subsequent agents.

Behavioural summary (b_u). A free-text description of the user's mobility personality, things like a preference for consistent routines versus spontaneous exploration, or visits concentrated in a small geographic radius versus spread across a wider area.

Preferred categories (c_u). A frequency-ranked list of the POI semantic categories the user actually visits most, giving the Reranker a compact categorical profile to lean on when it has to choose between candidates from different venue types.

Routine patterns (r_u). Daily and weekly visitation cycles, encoded as structured dictionaries. A user who keeps ending up at coffee shops on weekday mornings and transit hubs in the evenings has a strong enough routine that it already predicts a good chunk of what they will do next.

Exploration style (e_u). A ternary value $e_u \in \{\text{Low, Moderate, High}\}$, computed from the ratio of unique POIs to total check-ins. This field also plays a direct quantitative role in the retrieval weighting described in the following subsection.

Temporal habits (τ_u). A mapping from time-of-day and day-of-week slots to the venue categories that tend to show up in those slots; this is what makes time-sensitive predictions possible when the episode timestamps point at a particular temporal context.

Strategy hint (s_u). A meta-level reasoning directive put together from the five fields above, e.g., "prioritise transit and food venues on weekday mornings; deprioritise entertainment venues during work hours." The hint is injected straight into the Reranker's prompt, so the Reranker has less inference to do at prediction time.

Everything here is kept in natural-language form rather than as numeric vectors, so that every profile field can be dropped directly into an LLM prompt without an embedding or transformation step in between. That choice is part of why the system sits on top of any instruction-tuned LLM without retraining.

3.3.2 Retrieval Agent: Candidate Construction

Scoring all M POIs through an LLM at every prediction step is not feasible; M ranges from 5,086 to 13,564 across our datasets. So before inference starts, we put together a user-specific candidate set $\mathcal{C}_u \subseteq \mathcal{P}$ using two lightweight statistical signals.

Transition model. First-order Markov transition probabilities over consecutive POI pairs in the training corpus:

$$P_{\text{tr}}(q \mid p) = \frac{N(p,q)}{\sum_j N(p,j)}, \tag{3.11}$$

where $N(p,q)$ counts the number of $p \to q$ transitions across all users. To smooth the directional signal while still staying close to recency, we average scores over the user's L

most recent check-ins:

$$\bar{P}_{\mathrm{tr}}(q) = \frac{1}{L}\sum_{i=0}^{L-1} P_{\mathrm{tr}}(q \mid p_{n-i}), \qquad L = 2. \tag{3.12}$$

User preference model. Next, the normalised historical visit frequency for user u:

$$P_{\mathrm{pref}}(q) = \frac{N_u(q)}{|\mathcal{T}_u^H|}, \tag{3.13}$$

which keeps around places that matter personally to the user, even when the population-level transition pattern does not particularly recommend them.

Profile-conditioned fusion. The right balance between the two signals is not the same for every user; it depends on their exploration style. Low-exploration users revisit familiar places a lot, so the preference model is the more predictive signal for them. High-exploration users turn up at new places often, so the transition model is more useful in their case. A single LLM call over $\mathbf{P}_u$ produces adaptive fusion weights:

$$(w_{\mathrm{tr}},\ w_{\mathrm{pref}}) = \mathcal{F}_{\mathrm{wt}}(\mathbf{P}_u), \qquad w_{\mathrm{tr}} + w_{\mathrm{pref}} = 1. \tag{3.14}$$

The composite score and resulting candidate set are:

$$\mathrm{score}_u(q) = w_{\mathrm{tr}}\bar{P}_{\mathrm{tr}}(q) + w_{\mathrm{pref}}P_{\mathrm{pref}}(q), \tag{3.15}$$

$$\mathcal{C}_u = \text{Top-}K(\mathrm{score}_u(q))\,, \qquad K = 125. \tag{3.16}$$

Note that $\mathcal{C}_u$ is fixed before inference and does not change across episodes. The choice of $K = 125$ and $L = 2$ is justified empirically in Section 5.4.

3.3.3 Memory Agent: Episodic Consolidation

The Memory Agent is responsible for the episodic memory $\mathcal{M}_u^{(e)}$, which starts out empty as $\mathcal{M}_u^{(0)} = \varnothing$ at the beginning of Episode 1. After each episode it folds the ten newly observed check-ins from $\mathcal{E}_u^{(e)}$ into the existing summary:

$$\mathcal{M}_u^{(e)} = \mathcal{F}_{\mathrm{mem}}\Big(\mathcal{M}_u^{(e-1)},\ \mathcal{E}_u^{(e)}\Big)\,. \tag{3.17}$$

Each update is a single natural-language paragraph that blends what is already known about the user's test-session behaviour with whatever patterns show up in the most recent ten check-ins. It points out changes in the categories being visited relative to the historical baseline, any new spatial areas that the historical trajectory did not cover, and any deviations from the routines recorded in the identity prior. The output length is capped so the memory cannot just keep growing without limit.

One decision worth highlighting here is that the update does not depend on whether the last prediction was correct. The Memory Agent only sees the actual observed check-ins; it never sees the system's prediction. Its job is to track the user, not to judge the system. Thanks to that separation, behavioural evidence is always folded in, whether the preceding prediction hit or missed.

3.3.4 Reflection Agent: Metacognitive Reasoning

At the end of each episode, the Reflection Agent compares the system's prediction against the ground truth and updates the reflection record, starting from an empty $\mathcal{R}_u^{(0)} = \varnothing$:

$$\mathcal{R}_u^{(e)} = \mathcal{F}_{\text{ref}}\Big(\mathcal{R}_u^{(e-1)},\ \hat{p}_u^{(e)},\ y_u^{(e)}\Big)\,. \tag{3.18}$$

When the prediction was right, the reasoning strategy that produced it is reinforced, and the record starts to keep track of which approaches, paying more attention to temporal context, say, or trusting transition signals over raw frequency counts, keep working for this particular user.

When the prediction was wrong, a diagnostic is triggered instead. The agent looks at how the predicted POI relates to the ground truth (category mismatch, geographic proximity, temporal context, and so on) and names the specific failure mode. The diagnosis is written out as an imperative directive, e.g., "the model ranked candidates from the Office category above candidates from the Food category, but this user visits food venues during the late afternoon on weekdays." That directive is then injected into the next Reranker prompt, which adjusts, directly, which signals the Reranker is supposed to weigh more heavily.

Memory and reflection are not two versions of the same thing. Memory enriches what the Reranker knows about the user; reflection changes *how* the Reranker reasons, by giving it direct guidance on which strategies to apply and which to steer clear of. The ablation in Section 5.2 confirms that removing each of these components produces a qualitatively different pattern of degradation.

3.3.5 Reranker Agent: Inference and Integration

At each episode e, the Reranker pulls its inputs together to produce the prediction:

$$\hat{p}_u^{(e)} = \mathcal{F}_{\text{rank}}\Big(\mathcal{E}_u^{(e)-},\ \mathcal{S}_u^{(e-1)},\ \mathcal{C}_u\Big)\,. \tag{3.19}$$

The observable part of the current episode, $\mathcal{E}_u^{(e)-}$, carries the most immediate context: the nine observable check-ins of the episode (categories, locations and timestamps). The held-out check-in $y_u^{(e)}$ is the prediction target, so it is deliberately not part of the Reranker's input. The prior cognitive state $\mathcal{S}_u^{(e-1)} = (\mathbf{P}_u, \mathcal{M}_u^{(e-1)}, \mathcal{R}_u^{(e-1)})$ contributes the stable identity prior, the episodic record of what has happened so far during the test session, and the metacognitive assessment of how well the system has been reasoning. Conditioning on $e-1$ rather than e is what keeps the causality strict: nothing from inside episode e can affect the episode-e prediction. The candidate set $\mathcal{C}_u$ keeps the model focused on discriminating between plausible options, instead of having to recall things from the full vocabulary.

The Reranker prompt asks the model to rank candidates by weighing temporal context, categorical preferences, spatial patterns and the corrective directives from the reflection record, using the strategy hint s_u as a pre-synthesised starting point. The top-ranked candidate is read off as $\hat{p}_u^{(e)}$. The full ranking over $\mathcal{C}_u$ that comes out of this process is written $\hat{\mathbf{r}}_u$ and is the object used when we compute the evaluation metrics in Section 4.5.

3.3.6 The Continual Episodic Inference Loop

Algorithm 1 formalises the complete inference procedure. Four properties are worth pulling out about this loop.

Temporal causality. All predictions condition on $\mathcal{S}_u^{(e-1)}$, which means no episode-e observation can influence the episode-e prediction. The ground truth $y_u^{(e)}$ only enters the system through $\mathcal{F}_{\text{ref}}$, strictly *after* the prediction has already been committed.

Progressive enrichment. By the time we reach Episode 3, the cognitive state has been through two full consolidation-and-reflection cycles. The Reranker at Episode 3, therefore, is working with noticeably richer contextual and diagnostic information than it had at Episode 1.

Algorithm 1 Continual Inference in CogMaPOI

Require: $\mathcal{E}_u^{(1)}, \mathcal{E}_u^{(2)}, \mathcal{E}_u^{(3)}$; $\mathbf{P}_u$; $\mathcal{C}_u$

$\mathcal{M}_u^{(0)} \leftarrow \varnothing$; $\mathcal{R}_u^{(0)} \leftarrow \varnothing$; $\mathcal{S}_u^{(0)} \leftarrow (\mathbf{P}_u, \mathcal{M}_u^{(0)}, \mathcal{R}_u^{(0)})$
for $e = 1, 2, 3$ **do**
  $\hat{p}_u^{(e)} \leftarrow \mathcal{F}_{\text{rank}}(\mathcal{E}_u^{(e)-}, \mathcal{S}_u^{(e-1)}, \mathcal{C}_u)$
  $\mathcal{M}_u^{(e)} \leftarrow \mathcal{F}_{\text{mem}}(\mathcal{M}_u^{(e-1)}, \mathcal{E}_u^{(e)})$
  $\mathcal{R}_u^{(e)} \leftarrow \mathcal{F}_{\text{ref}}(\mathcal{R}_u^{(e-1)}, \hat{p}_u^{(e)}, y_u^{(e)})$
  $\mathcal{S}_u^{(e)} \leftarrow (\mathbf{P}_u, \mathcal{M}_u^{(e)}, \mathcal{R}_u^{(e)})$
end for
Evaluate on $\hat{p}_u^{(3)}$

Parameter-free adaptation. Nowhere in the loop is a gradient computed or a model fine-tuned. All of the adaptation runs through natural-language state updates encoded in structured prompts, so the whole framework can be deployed without any training infrastructure.

Evaluation fairness. Only Episode 3 predictions count for metric computation. Episodes 1 and 2 play the role of cognitive warm-up cycles that build the state used at Episode 3; scoring them would not give a fair comparison with the single-shot baselines, which read the full 30-check-in window in a single forward pass without any iterative warm-up. In both regimes, the same 30 check-ins are observed before a prediction is committed, and in neither regime does the system get to see the ground truth for the episode being scored.

Chapter 4

Implementation Details

4.1 Hardware and Software

Every experiment reported here was run on a single NVIDIA RTX 2000 Ada GPU with 16 GB of VRAM, on an Ubuntu 22.04 machine running CUDA 12.1 and Python 3.10. The inference stack is built around PyTorch 2.1.0; we use Hugging Face `transformers` 4.40.0 for loading the models, `bitsandbytes` 0.43.0 for the quantisation, and `accelerate` 0.28.0 for device placement.

4.2 Model Configuration

The main backbone we rely on is Llama 3.1-8B-Instruct. The reason is fairly practical: the Profiler has to emit structured JSON, and the Reranker has to emit a ranked candidate list, and base models without instruction tuning do not reliably hold either format.

We load the model with 4-bit NF4 quantisation (FP16 compute dtype, double quantisation on), which brings the 8B-parameter model down to around 5.5 GB of VRAM and leaves comfortable headroom for the KV cache. Generation runs at temperature 0.2, with a cap of 200 output tokens per call. For every Reranker call we run two inference passes, average the ranked outputs across them, and pick the candidate with the lowest mean rank; this cuts the stochastic variance in the top-1 prediction in half for no real extra cost. The Memory and Reflection agents run a single pass each, since small variations in the phrasing of their outputs do not feed back into downstream prediction quality.

4.3 Datasets

We use three Foursquare check-in datasets taken from the CoMaPOI benchmark [2]. Each record carries a user ID, a POI ID, a semantic category, GPS coordinates and a timestamp. Summary statistics are in Table 4.1.

TABLE 4.1: Summary of key statistics across the three Foursquare datasets

Dataset	Users	POIs	Check-ins	Avg. check-ins/user	Period
NYC	988	5,086	99,964	101.2	Apr 2012 – Feb 2013
TKY	2,206	7,849	325,313	147.5	Apr 2012 – Feb 2013
CA	1,818	13,564	174,791	96.1	Feb 2009 – Oct 2010

NYC has high density in both urban and geographic terms. **TKY** has exceptionally regular trajectories, due to the nature of how people commute in this area because of the presence of trains; hence, it is the easiest for sequential prediction among the three datasets. **CA** is geographically distributed across multiple cities, and also has the largest number of points-of-interest (POIs) in its vocabulary of 13,564; hence, making the retrieval task more challenging.

4.4 Data Preprocessing and Episode Construction

As described by CoMaPOI [20], each user's check-ins are ordered chronologically, and the last 30 become the test trajectory, while everything prior to these 30 check-ins becomes the historical trajectory. Any user with less than 30 check-ins overall is eliminated from the dataset. According to our definition in Section 3.1.2, the test trajectory is divided into three episodes of ten check-ins each, where the 10th check-in of each episode serves as the ground truth.

4.5 Evaluation Metrics

We report three standard retrieval metrics, all computed only on the Episode 3 predictions (see Section 3.3.6), at cut-offs $k \in \{5, 10\}$ per the CoMaPOI protocol.

HR@k (Hit Rate) is the fraction of users for whom the ground truth shows up inside the top-k of the returned list:

$$\mathrm{HR@}k = \frac{1}{|\mathcal{U}|} \sum_{u} \mathbf{1}\left[y_u^{(3)} \in \text{Top-}k(\hat{\mathbf{r}}_u)\right]. \tag{4.1}$$

NDCG@k discounts correct answers by rank position:

$$\mathrm{NDCG@}k = \frac{1}{|\mathcal{U}|} \sum_{u} \frac{\mathbf{1}\left[\mathrm{rank}(y_u^{(3)}) \leq k\right]}{\log_2(\mathrm{rank}(y_u^{(3)}) + 1)}. \tag{4.2}$$

MRR (Mean Reciprocal Rank) assigns each user $1/\mathrm{rank}(y_u^{(3)})$:

$$\mathrm{MRR} = \frac{1}{|\mathcal{U}|} \sum_{u} \frac{1}{\mathrm{rank}(y_u^{(3)})}. \tag{4.3}$$

HR@k is a pure coverage metric: any placement in the top-k counts, so it says nothing about the quality of the ordering inside that window. NDCG@k does pay attention to that ordering and gives more credit to correct answers placed higher. MRR is the most discriminative of the three for systems that are trying to push the correct POI to rank 1, a correct answer at rank 2 contributes 0.5, and one at rank 3 contributes 0.33. The gap between MRR and HR that shows up in the results (Chapter 5) is a reflection of the fact that CogMaPOI's main effect is on rank quality, not on raw coverage.

4.6 Baselines and Comparison Protocol

We compare CogMaPOI against thirteen baselines split across five categories: transformer-based (SASRec [1], BERT4Rec [2], TiCoSeRec [3]), graph-based (GETNext [5], CrossDR [6], MAERec [7], DuoRec [4]), generative (POIGDE [10], MTNet [8], DiffuRec [9]), LLM-based (LLM4POI [11]), and multi-agent (CoMaPOI [20], LMAPOI [21]). Every baseline number is pulled directly from the CoMaPOI unified evaluation benchmark [20], which uses the same data splits, candidate sets and evaluation cut-offs that we do. Before any comparative run, the hyperparameters $K = 125$ and $L = 2$ were fixed from a retrieval analysis on NYC alone, and the same values are applied uniformly across all three datasets afterwards.

Chapter 5

Results and Analysis

This chapter takes the experimental results apart from four different angles: comparative performance against the thirteen baselines (Section 5.1), controlled ablations that pull apart the individual contributions of episodic memory and metacognitive reflection (Section 5.2), backbone sensitivity across a range of LLM sizes (Section 5.3), and a retrieval-hyperparameter analysis (Section 5.4). Section 5.5 then tries to pull the cross-cutting patterns together.

5.1 Comparative Performance Analysis

Table 5.1 carries the full comparative results. Before we get into per-dataset specifics, there is one feature of the table that is worth calling out up front: a consistent asymmetry between the metrics. MRR gains are quite a lot larger than HR gains, and this holds across all three datasets. Making sense of that asymmetry turns out to be central to understanding what CogMaPOI actually does.

5.1.1 The MRR-versus-HR Asymmetry

On NYC, MRR goes up by 56.88% over the strongest baseline, while HR@10 goes up only 8.28%. On TKY, MRR rises by 46.36%, with HR@10 barely moving at 2.00%. On CA, the divergence is most extreme: MRR gains 89.35% while HR@10 lifts by 44.72%. None of this is incidental; it tracks very directly with what the cognitive adaptation mechanisms are designed to do.

TABLE 5.1: Performance comparison (%). N@k = NDCG@k. denotes relative improvement over the best baseline. NYC TKY CA Model HR@5 HR@10

	NYC					TKY					CA				
Model	HR@5	HR@10	N@5	N@10	MRR	HR@5	HR@10	N@5	N@10	MRR	HR@5	HR@10	N@5	N@10	MRR
SASRec	40.38	48.28	28.98	31.26	27.01	34.72	43.25	25.48	28.37	24.70	21.34	26.40	16.04	17.68	15.94
BERT4Rec	39.60	46.70	29.03	30.94	27.11	36.17	43.47	26.70	29.08	25.39	23.22	28.22	17.26	18.66	16.61
DuoRec	34.72	40.69	24.54	26.34	22.76	24.66	31.19	18.77	20.89	18.67	19.47	24.04	14.47	15.85	14.31
TiCoSeRec	40.69	48.08	29.65	32.04	27.54	29.33	37.04	21.83	24.21	21.51	22.66	28.11	16.99	18.48	16.75
CrossDR	40.80	47.20	30.67	32.59	28.66	32.51	40.57	23.58	26.19	22.65	22.11	28.61	16.11	18.18	15.79
MAERec	37.65	44.53	27.72	29.85	26.10	32.73	40.66	23.93	26.14	23.05	24.37	29.37	17.76	19.48	17.09
GETNext	43.60	51.30	32.56	35.13	30.97	41.11	48.50	30.75	33.21	29.21	24.26	29.32	17.83	19.48	17.16
POIGDE	34.11	38.36	26.64	27.97	25.40	28.20	31.82	22.46	23.57	21.80	22.61	26.73	18.24	19.49	17.89
MTNet	47.57	53.24	35.85	37.69	33.40	44.15	51.22	32.99	35.37	31.18	30.25	36.08	22.39	24.27	21.35
DiffuRec	37.25	40.79	29.16	30.23	27.74	34.50	39.89	26.89	28.70	25.94	21.67	24.15	17.35	18.06	16.60
LLM4POI	44.64	46.66	37.49	38.16	35.90	34.90	39.48	27.23	28.22	29.72	23.65	25.91	19.01	19.39	20.13
CoMaPOI	51.62	59.01	40.42	42.82	37.67	45.83	54.26	34.48	37.20	31.82	33.00	39.16	24.96	26.96	23.10
LMAPOI	54.56	64.66	42.56	45.75	40.28	48.26	57.52	36.26	39.49	34.13	31.43	37.23	24.30	26.17	23.11
CogMaPOI	**64.67**	**70.01**	**51.97**	**53.76**	**63.19**	**48.67**	**58.67**	**37.13**	**40.44**	**49.95**	**46.67**	**56.67**	**33.60**	**36.85**	**43.76**
Δ (rel.%)	+18.53	+8.28	+22.11	+17.51	+56.88	+0.85	+2.00	+2.40	+2.41	+46.36	+41.42	+44.72	+34.62	+36.69	+89.35

HR@k treats every placement inside the top-k as equivalent. A system that puts the correct POI at rank 8 collects the same credit as one that puts it at rank 1, as long as $k \geq 8$. MRR, on the other hand, penalises every position below first via a reciprocal-rank score. The episodic loop does not change which candidates get retrieved, the candidate set is fixed before inference and stays the same across episodes. What the loop does change is how those candidates are ordered. Episodic memory hands the Reranker a richer, more up-to-date description of the user's behaviour, so it can weigh candidates more accurately. The reflection record goes one step further and corrects, directly, the ranking errors made in earlier episodes. Both of these mechanisms operate entirely within the retrieved set; they reshuffle how candidates are ordered, not which candidates show up. MRR happens to be the metric most sensitive to exactly that kind of improvement, which is why its gains come out consistently, and consistently larger, than the HR gains, across all three datasets and all five metrics we report.

5.1.2 Dataset-Level Analysis

The pattern of improvement across the three datasets is not uniform, and that variation itself says something about when the cognitive adaptation framework is most valuable.

On **NYC**, the gains are spread across the metrics fairly evenly. CogMaPOI ends up at an MRR of 63.19%, against 40.28% for the strongest baseline (LMAPOI), which is a relative lift of 56.88%. The NDCG numbers tell a similar story: NDCG@5 rises from 42.56% to 51.97%, NDCG@10 from 45.75% to 53.76%, both of those translating into relative gains of 22.11% and 17.51% respectively. The NDCG results are useful here because they confirm that the improvement is not concentrated at rank 1 alone; the cognitive state nudges the

correct POI toward the top of the list in a graded way, so both rank-1 placement (what MRR captures) and top-5 placement (what NDCG@5 captures) improve together. The prior multi-agent systems manage competitive HR numbers because their decomposition strategy already retrieves the correct POI reasonably often; what they fall down on is MRR and NDCG, which is another way of saying that they retrieve the right candidate but consistently rank it in the wrong slot. CogMaPOI's reflection record is built precisely to correct that failure mode.

On **TKY**, the structural regularity of the dataset puts a ceiling on HR and NDCG improvements but leaves plenty of room for MRR to move. The sequential patterns in Tokyo commuter trajectories are strong enough that transition-based retrieval already does a decent job of putting the correct POI into the candidate set, and that high coverage compresses how much room there is for HR and NDCG gaps between methods. Yet MRR still lifts by 46.36%, which is the second largest gain across the datasets. The takeaway is that even in highly regular trajectory settings, the static models commit consistent ranking errors that episodic context and reflective correction can still address. When the correct POI is almost always present but almost always mis-ranked, the reflection mechanism's targeted corrective guidance is precisely where the leverage is.

On **CA**, we see the largest improvements, and unlike on NYC and TKY they show up on every one of the five metrics simultaneously: HR@5 is up 41.42%, NDCG@5 by 34.62%, NDCG@10 by 36.69%, and MRR by 89.35%. CA has 13,564 unique POIs spread across a geographically dispersed region, a vocabulary nearly three times the size of NYC's. When the spatial heterogeneity is that large, user-level individuation is really the main source of predictive signal left to exploit. The identity prior $\mathbf{P}_u$ is exactly where we record the sort of user-specific geographic and categorical preferences that tell one user apart from the broader population, and that signal becomes more discriminative the bigger the search space gets. Episodic memory adds on top of this because behavioural drift matters more in precisely those settings where the set of plausible next destinations is wide to begin with. The NDCG and MRR gains on CA together suggest that the cognitive state is doing more than helping the system identify the correct POI, it is also pushing it substantially higher up the ranked list than any baseline manages.

5.2 Ablation Study

Table 5.2 shows the ablation results on NYC. When we remove each of the components, the degradation patterns that come out are not only quantitatively different; they are

Table 5.2: Ablation on NYC (%). Relative decrease from CogMaPOI in parentheses.

Configuration	HR@5	HR@10	NDCG@5	NDCG@10	MRR
CogMaPOI (full)	64.67	70.01	51.97	53.76	63.19
w/o Memory	60.02 (−7.19%)	68.09 (−2.74%)	47.33 (−8.93%)	49.84 (−7.29%)	57.96 (−8.27%)
w/o Reflection	58.01 (−10.30%)	69.15 (−1.23%)	48.65 (−6.39%)	52.98 (−1.45%)	57.12 (−9.61%)
w/o Both	51.76 (−19.97%)	63.18 (−9.75%)	38.81 (−25.32%)	41.59 (−22.64%)	50.66 (−19.83%)

qualitatively different, and that qualitative difference is at least as informative as the size of the drop.

5.2.1 Memory as Broad Contextual Enrichment

Taking the episodic memory out produces a proportional decline that is spread broadly across all five metrics: HR@5 drops 7.19%, HR@10 drops 2.74%, NDCG@5 drops 8.93%, NDCG@10 drops 7.29%, and MRR drops 8.27%. That near-uniform spread is itself informative. If memory were a mechanism that specifically improved top-1 placement, we would expect the MRR hit to be disproportionately larger than the others; if it specifically helped top-5 placement, we would expect HR@5 and NDCG@5 to suffer disproportionately. Neither of those is what the ablation shows. The breadth is what memory's role actually looks like, it works as a broad context enrichment layer that helps the Reranker reason better across the whole ranked list, not at any one rank depth.

When the memory is gone, the Reranker at Episode 3 is left with just the current ten check-ins and the static identity prior. The identity prior encodes historical preferences, and those may no longer match the user's present situation. A user whose current session has drifted away from their historical baseline, into a different part of the city, or into a different category of venue, sends no corrective signal to a system that cannot accumulate what it is observing. Memory closes that hole by keeping a running description of test-session behaviour that is rewritten, unconditionally, after every episode.

5.2.2 Reflection as a Top-Rank Corrector

Removing reflection, by contrast, produces a concentrated pattern with a clear asymmetry between shallow and deep ranking metrics. HR@5 drops 10.30% and MRR drops 9.61%, both real-sized falls, while HR@10 drops only 1.23% and NDCG@10 only 1.45%. NDCG@5 falls 6.39%, enough to confirm that top-5 quality is meaningfully affected, but everything deeper in the ranked list holds fairly stable.

There is a precise reading of this pattern. HR@5 and HR@10 only differ in whether the ground-truth POI lands inside positions 1–5 or inside positions 1–10. NDCG@5 and NDCG@10 differ in how much credit is given to positions in the lower half of the top-10, with NDCG@5 putting more weight on the very top. A large fall in HR@5 and NDCG@5 combined with a minimal fall in HR@10 and NDCG@10 therefore means that taking reflection out moves the correct POI from the top-5 into positions 6–10. The candidate is still being retrieved, the Reranker just is not promoting it into the positions that really matter for ranking quality.

That is exactly the failure mode the Reflection Agent is designed to fix. It diagnoses specific ranking errors from earlier episodes, things like routinely placing the wrong category at the top, or over-weighting raw venue frequency compared to time-of-day context, and emits targeted directives for the next Reranker call. Without that corrective signal the system falls back to a ranking strategy that is plausible in general but poorly tuned to the individual user, which gives the concentrated degradation in HR@5 and NDCG@5 we see in the ablation.

5.2.3 Synergistic Effect of Memory and Reflection

Taking both components out at the same time produces degradations that exceed the sum of the individual losses for most metrics. NDCG@5 falls 25.32% jointly, against individual drops of 8.93% and 6.39% that would only add up to 15.32%. NDCG@10 falls 22.64% jointly versus a naive sum of 8.74%. MRR drops 19.83% jointly versus 17.88% cumulatively. HR@5 likewise falls 19.97% jointly, which is also higher than 7.19% and 10.30% put together.

The reason is that memory and reflection are not statistically independent in their effects. How good the reflection diagnosis can get is bounded by how much information the Reranker had to reason with when it made the error being diagnosed in the first place. Richer episodic context means the Reranker was reasoning from better

Table 5.3: Backbone sensitivity on NYC (%).

Backbone	HR@5	HR@10	NDCG@5	NDCG@10	MRR
Llama 3.1-8B	**64.67**	**70.01**	**51.97**	**53.76**	**63.19**
Mistral 7B	62.07	67.76	49.63	50.94	60.08
Qwen 2.5-5B	61.26	67.18	49.18	50.35	59.35
Qwen 1.5-7B	60.43	66.72	48.69	49.71	58.92
Llama 3.2-3B	59.85	65.54	47.65	48.22	57.81
Qwen 2.5-3B	58.09	62.91	45.78	46.57	55.99

information, and that in turn makes it possible for reflection to give a more precise account of why the prediction still went wrong. When both components are removed, the system loses not just each form of adaptation on its own, but also the reinforcing relationship between them. The resulting degradation, especially on the NDCG metrics, which penalise both depth and rank position jointly, is noticeably larger than what either removal would predict on its own. The joint ablation, in short, shows that memory and reflection are coupled components of the adaptive reasoning process rather than two separate features.

5.3 LLM Backbone Sensitivity

The backbone numbers in Table 5.3 follow a smooth gradation; performance deteriorates as the backbone capacity decreases, but it does so steadily and not abruptly. The smallest model of the two, Qwen 2.5-3B, shows 58.09% in HR@5, 45.78% in NDCG@5 and 55.99% MRR. Even though those are the smallest numbers among the backbones in question, each of them still offers a solid improvement compared to the LMAPOI model, which represents our best static baseline with HR@5 of 54.56%, NDCG@5 of 42.56% and MRR of 40.28%. Moreover, even the weakest version of CogMaPOI model provides an improvement over the best previous multi-agent model in all three metrics, ranging from 3 to 15 percentage points.

Two comparisons inside the table are particularly informative, because they let us tease apart the effect of instruction-following alignment from the effect of raw parameter count. Qwen 1.5-7B scores HR@5 of 60.43% and NDCG@5 of 48.69%, both *below* Qwen 2.5-5B (HR@5: 61.26%, NDCG@5: 49.18%), in spite of having more parameters. The Qwen 2.5 training pipeline introduced a meaningful improvement in alignment, and that matters here because every agent in CogMaPOI leans on the model's ability to follow structured output specifications, the Profiler has to emit valid JSON, the Reranker has to emit a ranked list,

and the Reflection Agent has to distinguish correct from incorrect prediction scenarios and format its diagnosis accordingly. A model that keeps breaking those formats damages every downstream component it feeds into. The gap between Mistral 7B and Llama 3.1-8B tells the same story: a 3.86% difference in HR@5 and 4.93% in MRR between models of comparable size, and the difference is almost certainly about the stronger instruction tuning of Llama 3.1-8B-Instruct rather than anything to do with raw capacity.

5.4 Retrieval Stage Analysis

Both of the retrieval hyperparameters are selected based on the final ranking metrics rather than on retrieval recall alone. The reason is that a candidate set with high recall but a poor internal ordering can drive HR up while simultaneously pulling MRR down, so optimising on recall alone gives the wrong answer. The analysis in this section uses NYC.

5.4.1 Candidate Set Size

Recall values in Table 5.4 level off from K=75 onwards. In other words, together the Markov transition model and the user preference model pick up a fixed fraction of ground-truth POIs as high-scoring candidates, and pushing the candidate set any larger does not change that fraction.

Inside this plateau, the ranking metrics keep moving, and where they move is the interesting part. Both NDCG@10 and MRR rise from K=75 to K=125, which means that adding candidates in this range genuinely introduces useful lower-probability correct options without yet adding enough distractors to confuse the Reranker. However, for values beyond K=125 the pattern changes direction. For K=150, both MRR and NDCG@10 decrease to 61.15% and 51.65%, respectively, lower than the values of 63.19% and 53.76% at K=125. Finally, even though HR@10 improves to 69.84% at K=200, MRR continues to struggle at 62.72%. This is reminiscent of the asymmetrical relationship between MRR and HR observed in the comparative experiments. In other words, adding more ground-truth points to the top-10 list does not make the rankings any better.

The results of using K=125 also merit closer inspection. MRR is still a fairly respectable 60.87% at this value, but almost 36% of the users have no valid ground-truth POI candidates within the list and therefore are removed from the MRR computation. This

TABLE 5.4: Effect of candidate set size K on NYC (L=2, %).

K	Recall@K	HR@5	HR@10	NDCG@5	NDCG@10	MRR
25	64.07	61.65	63.29	49.74	50.12	60.87
50	68.12	64.67	68.00	50.69	51.75	61.29
75	70.04	64.67	68.67	50.99	52.37	61.97
100	70.04	64.67	69.33	50.72	52.20	61.42
125	**70.04**	**64.67**	**70.01**	**51.97**	**53.76**	**63.19**
150	70.04	63.34	68.00	50.41	51.65	61.15
200	70.04	64.25	69.84	51.81	53.66	62.72

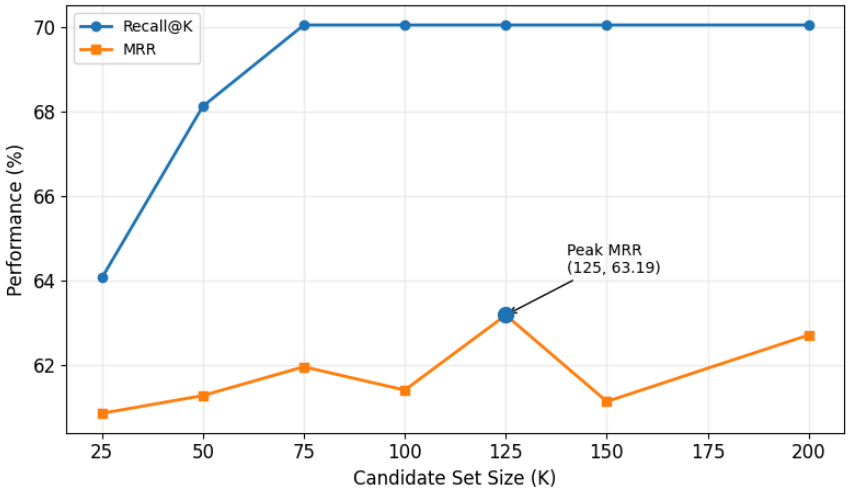

FIGURE 5.1: Recall@K and MRR vs. candidate set size K (NYC, L=2).

makes the number a biased sub-set score rather than a comprehensive metric. NDCG@10 at K=125, including all users, falls to 50.12

5.4.2 Transition Window Size

Table 5.5 shows recall rising monotonically with L, while ranking quality actually peaks at L=2 and then drops for larger windows. Averaging transition scores over more of the recent check-ins surfaces a broader set of candidates, but older transitions carry less directional signal about where the user is heading right now. As the window grows, the candidate set becomes more inclusive, but at the cost of bringing in candidates whose relevance is harder for the Reranker to pin down.

The L=4 row shows that tension very clearly. It has the highest HR@5 in the table at 65.34%, meaning it recovers the most correct POIs into the top-5. Yet its MRR is the lowest at 59.60%, and NDCG@10 also slips relative to L=2. So it is not that the broader

TABLE 5.5: Effect of transition window size L on NYC (K=125, %).

L	Recall@K	HR@5	HR@10	NDCG@5	NDCG@10	MRR
1	68.02	63.94	67.15	51.61	52.76	62.16
2	**70.04**	**64.67**	**70.01**	**51.97**	**53.76**	**63.19**
3	70.04	64.67	68.00	50.93	51.96	61.21
4	72.06	65.34	67.33	50.58	51.19	59.60
5	74.19	64.01	65.33	50.58	50.98	60.73

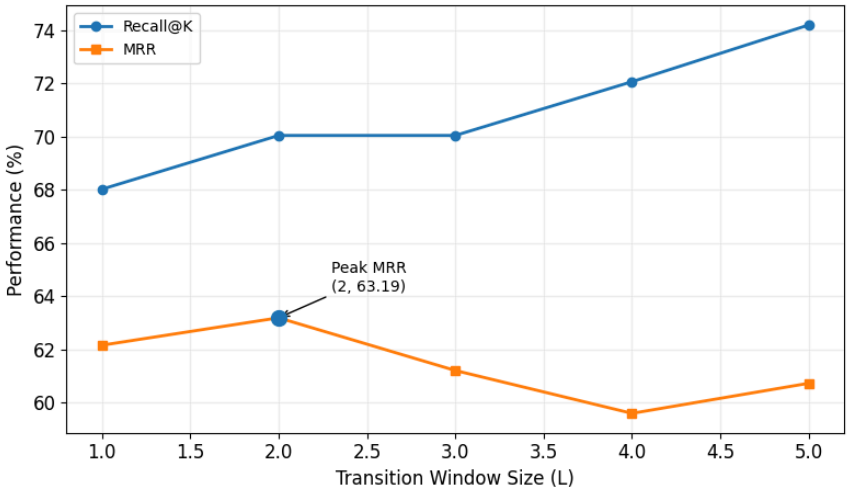

FIGURE 5.2: Recall@K and MRR vs. transition window size L (NYC, K=125).

candidate pool quietly shifts performance from top-1 to a wider list, it actively degrades position-weighted quality. L=2 is the only configuration that simultaneously maximises NDCG@10 and MRR while also hitting recall saturation, which is why it is the selection that best serves both ranking precision and position-weighted accuracy.

5.5 Discussion and Key Takeaways

Three cross-cutting observations tie the four experiments together.

The consistent MRR-versus-HR asymmetry in the comparative results is a predictable consequence of what the cognitive adaptation loop is designed to do. Candidate retrieval is static here; the episodic loop works entirely on the reranking side. So the loop's contribution *should* show up most strongly on metrics that are sensitive to rank ordering, and that is what happens across all three datasets. This asymmetry is not a limitation of the system. It reflects where the episodic components are placed within the architecture.

The ablation results show that memory and reflection serve different roles rather than acting as two similar mechanisms. When memory is removed, there is a consistent drop across all five metrics, which matches its role. It supports the overall quality of the Reranker's contextual reasoning across the full ranked list. In contrast, removing reflection affects HR@5 and MRR the most, with drops of 10.30% and 9.61%. HR@10 and NDCG@10 change very little. This aligns with its role, which is to correct errors that prevent the correct POI from reaching the top positions, rather than affecting whether it appears in the list. When both components are removed together, the drop in NDCG and MRR is larger than the sum of the individual drops, showing that their effects reinforce each other.

The backbone analysis further suggests that the advantage of the framework comes from how the cognitive state is designed, rather than from the specific backbone model. Performance does decrease as the backbone becomes smaller, but the drop is moderate. Even the smallest configuration tested still performs better than the strongest prior system by a clear margin. Within the tested range, instruction-following alignment explains more of the performance differences than parameter count. This indicates that alignment quality is likely more important than model size when selecting a backbone for this framework.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis has introduced CogMaPOI, a five-agent LLM framework that treats next-POI recommendation as continual episodic inference instead of static single-pass prediction. The motivating claim is a fairly simple one: human spatial decision-making leans on three distinct cognitive faculties at once, stable long-term preferences built up over time, an awareness of recent activity and short-term behavioural shifts, and the ability to recognise and correct one's own reasoning errors. Every prior system we looked at in the literature, from deep sequential models and graph-based encoders to LLM-based prompt approaches and the latest multi-agent frameworks, picks up at most the first of these. CogMaPOI, by contrast, carries all three through three separately maintained state components, each assigned to a dedicated agent inside a strictly causal inference loop.

The first contribution is the formulation of next-POI prediction as continual episodic inference. Instead of producing a prediction from a fixed representation and then stopping, the system carries a cognitive state that evolves through natural-language updates as new observations come in. This formulation needs no gradient computation, no parameter modification and no task-specific fine-tuning at any stage, which makes it drop-in on top of any instruction-tuned language model.

The second contribution is the five-agent architecture that actually implements this formulation. The Profiler turns the historical trajectory into a structured identity prior, encoded across six semantically meaningful fields. The Retrieval Agent assembles a user-specific candidate set by mixing Markov transition probabilities with personalised

visit frequency, weighted adaptively by the user's exploration profile. The Memory Agent keeps a running summary of observed test-session behaviour, rewritten unconditionally after every episode. The Reflection Agent diagnoses prediction outcomes, reinforces successful strategies, and issues corrective directives when predictions miss. The Reranker integrates the aggregated state at each episode to generate the final ranking output.

The third contribution is empirical evaluation on three Foursquare datasets against 13 baselines, including transformers, graph networks, generative models, LLMs, and multi-agents. Our CogMaPOI model attains relative MRR gains of 56.88%, 46.36% and 89.35% for NYC, TKY, and CA, respectively, compared to the best-performing baselines under each metric. The findings from our ablation experiments show that memory and reflection work in a complementary and non-redundant manner, where memory enlarges the context for reasoning, while reflection enhances the ranking precision at top ranks. Even the smallest architecture we consider, namely the 3B parameter model, outperforms the best-performing static model across all metrics.

6.2 Future Work

The results and the architectural analysis point to several directions in which this work can be meaningfully extended.

The retrieval stage, as it stands, uses two statistical signals. One is first-order Markov transition probabilities, and the other is historical visit frequency. These signals reflect patterns already present in the training data. At the same time, they miss cases that are not well represented there. Venues visited for reasons that are not reflected in sequential history, a first-time discovery, an event-driven detour, a place reached through a social recommendation, fall below the scoring threshold and never make it to the Reranker. Since the retrieval stage currently recovers about 70% of the ground-truth POIs, there is a real structural ceiling imposed by the retrieval design. Semantic similarity retrieval, using dense representations of POI descriptions and user profiles, could help pick up such venues by matching on conceptual relevance rather than co-occurrence frequency. Geographic proximity retrieval could pull in spatially adjacent POIs that would not otherwise score highly. A two-stage pipeline, with fast statistical retrieval at the first stage and semantic re-scoring at the second, is a natural direction for raising candidate quality without giving up the efficiency of the current design.

The evaluation window spans three episodes by design, which lines the setup up with the CoMaPOI benchmark and keeps the comparison with prior systems direct. What that design does not tell us is whether the gains compound with more warm-up cycles or plateau. Running the episodic loop over five or ten episodes would clarify whether the reflection record keeps improving predictions as more diagnoses accumulate, or whether it saturates once the main failure modes for a given user have been identified and corrected. Longer sequences would also give us a cleaner read on how well the Memory Agent tracks sustained behavioural drift, rather than just short-term deviations.

The reflection mechanism, at the moment, lives strictly inside a single user's test session. In practice, users with similar mobility profiles and exploration styles tend to trip the Reranker up in similar ways, which points toward a possible form of cross-user knowledge transfer. Reflection patterns taken from users with well-characterised profiles could pre-populate the initial reflection state of new users with comparable histories, so that the system starts from an informed corrective baseline rather than an empty one. This would be most useful in cold-start scenarios, where the historical trajectory is too sparse to build a reliable identity prior and the system has to borrow something from similar users.

Bibliography

[1] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)*, 2018.

[2] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019.

[3] Y. Dang, E. Yang, G. Guo, L. Jiang, X. Wang, X. Xu, Q. Sun, and H. Liu, "TiCoSeRec: Augmenting data to uniform sequences by time intervals for effective recommendation," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[4] R. Qiu, Z. Huang, H. Yin, and Z. Wang, "Contrastive learning for representation degeneration problem in sequential recommendation," in *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*, 2022.

[5] S. Yang, J. Liu, and K. Zhao, "GETNext: Trajectory flow map enhanced transformer for next POI recommendation," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

[6] H. Tao, J. Zeng, Z. Wang, M. Gao, and J. Wen, "Next POI recommendation based on spatial and temporal disentanglement representation," *IEEE International Conference on Web Services*, 2023.

[7] Y. Ye, L. Xia, and C. Huang, "Graph masked autoencoder for sequential recommendation," in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

[8] T. Huang, X. Pan, X. Cai, Y. Zhang, and X. Yuan, "Learning time slot preferences via mobility tree for next POI recommendation," in *Proceedings of the Thirty-Eighth*

AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, 2024.

[9] Z. Li, A. Sun, and C. Li, "DiffuRec: A diffusion model for sequential recommendation," *ACM Transactions on Information Systems*, 2023.

[10] Y. Yang, S. Zhou, H. Weng, D. Wang, X. Zhang, D. Yu, and S. Deng, "Siamese learning based on graph differential equation for next-poi recommendation," *Applied Soft Computing*, 2024.

[11] P. Li, M. de Rijke, H. Xue, S. Ao, Y. Song, and F. D. Salim, "Large language models for next point-of-interest recommendation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[12] J. Kim, Y. Seo, Y. Kim, J. Kang, J. Shin, P. Trirat, and J.-G. Lee, "Large language models are zero-shot point-of-interest recommenders," *Data Mining and Knowledge Discovery*, 2025.

[13] D. Wang, Y. Huang, S. Gao, Y. Wang, C. Huang, and S. Shang, "Generative next POI recommendation with semantic ID," in *KDD '25: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025.

[14] J. Chen, H. Wang, J. Shang, and Chaomurilige, "Interpretable embeddings for next point-of-interest recommendation via large language model question-answering," *Mathematics*, 2024.

[15] T. Wang and C. Wang, "Embracing large language models for point-of-interest recommendations," *IEEE Intelligent Systems*, 2024.

[16] ——, "Point-of-interest recommendations and large language models: A powerful combination," *IEEE Intelligent Systems*, 2025.

[17] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[18] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, 2025.

[19] Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang, "MACRec: A multi-agent collaboration framework for recommendation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[20] L. Zhong, L. Wang, X. Yang, and Q. Liao, "CoMaPOI: A collaborative multi-agent framework for next POI prediction bridging trajectory and language," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025.

[21] S. Gao, Z. Wang, and Y. Zhao, "LMAPOI: Large language model based multi-agent framework for next POI recommendation," *IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, 2025.

[22] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[23] F. Sarfraz, E. Arani, and B. Zonooz, "Error sensitivity modulation based experience replay: Mitigating abrupt representation drift in continual learning," in *11th International Conference on Learning Representations (ICLR 2023)*. OpenReview, 2023.

[24] M. Boeltzig, N. Liedtke, S. Siestrup, F. Mecklenbrauck, M. F. Wurm, I. Bramão, and R. I. Schubotz, "The benefit of being wrong: How prediction error size guides the reshaping of episodic memories," *NeuroImage*, 2025.

[25] F. Pupillo, J. Ortiz-Tudela, R. Bruckner, and Y. L. Shing, "The effect of prediction error on episodic memory encoding is modulated by the outcome of the predictions," *npj Science of Learning*, 2023.

[26] G. Schillaci, U. Schmidt, and L. Miranda, "Prediction error-driven memory consolidation for continual learning," *KI: Künstliche Intelligenz*, 2020.

Chilukala Amrutha

CogMaPOI: A Cognition-Inspired Multi-Agent Framework for Continual Next POI Recommendation with Episodic Memory a...

Indian Institute of Information Technology Design And Manufacturing, Kurnool

Document Details

Submission ID

trn:oid:::3618:136202437

Submission Date

Apr 23, 2026, 3:21 AM GMT+5:30

Download Date

Apr 23, 2026, 3:42 AM GMT+5:30

File Name

CogMaPOI_Thesis.pdf

File Size

1.3 MB

54 Pages

15,043 Words

79,329 Characters

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Cited Text
- Abstract

Match Groups

51 Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
3% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.